

{
blueprint_version: "v29",
title: "Nexus Blueprint V29 - Local First Hybrid Multi-Agent",
metadata: {
author: "Iswan (Nexus Folder)",
created_at: "2025-12-06T18:52:00+07:00",
mode: "local-first hybrid",
notes: "Lokal dominan; cloud only backup; server raksasa as manual fallback; secure credentials offline."
},
doctrine: {
principles: [
Lokal-first: inferensi cepat di Termux/HP,
Cloud as backup: manual sync only via rclone,
Server fallback: manual trigger, strict quota,
Least privilege: API keys encrypted locally,
Audit & retention: logs archived to Drive folder Backup_Platform_AI/logs
],
thresholds: {
local_latency_ms: 1000,
server_fallback_latency_ms: 3000,
max_termux_usage_gb: 5
}
},
environment_paths: {
termux_home: "/data/data/com.termux/files/home",
work_dir: "~/JEQ_WORK",
bin_dir: "~/JEQ_WORK/bin",

logs_dir: "~/JEQ_WORK/logs",
env_dir: "~/JEQ_WORK/env",
sdcard_models: "/sdcard/AI_Models_User",
nexus_drive_folder: "gdrive:Backup_Platform_AI"
},
agents: [
{
name: "Agent_ZERO",
type: "local_dispatcher",
repo_example: "github:example/agent-zero-ai",
role: "Primary local decision maker; routes prompts to local model or fallback",
config: {
latency_threshold_ms: 1000,
model_priority: ["phi-3-q4", "tiny-llama-q4"],
log_level: "INFO"
}
},
{
name: "Agent_SYNC",
type: "cloud_sync",
repo_example: "github:example/agent-sync",
role: "rclone-based sync agent; pushes logs and blueprint to Nexus Drive",
config: {
remote: "gdrive",
remote_path: "Backup_Platform_AI",
sync_policy: "manual_or_on_wifi"
}

},
{
name: "Agent_QAOA",
type: "quantum_orchestrator",
repo_example: "github:example/agent-qaoa",
role: "Invokes Qiskit simulator for optimization tasks; returns structured results",
config: {
simulator: "qasm_simulator",
max_shots: 1024
}
},
{
name: "Agent_MONITOR",
type: "watchdog_agent",
repo_example: "github:example/agent-watchdog",
role: "System watchdog and health reporter; restarts local model on failure",
config: {
check_interval_sec: 60,
restart_on_fail: true,
max_restarts_per_hour: 3
}
},
{
name: "Agent_FALLBACK",
type: "server_orchestrator",
repo_example: "github:example/agent-fallback",
role: "Manual gateway to server raksasa APIs; requires encrypted API key",

config: {
enabled: false,
manual_trigger_file: "~/JEQ_WORK/env/server_call.trigger"
}
}
],
components: {
watchdog: {
description: "Lightweight watchdog running in Termux to monitor processes, disk, memory, and model responsiveness.",
install_cmds: [
pkg install -y busybox,
pip install --no-cache-dir psutil
],
config: {
script_path: "~/JEQ_WORK/bin/watchdog.sh",
checks: ["process:llama_main", "disk_usage:<80%", "memory_free_mb:500"],
actions: {
on_process_down: "restart_llama_main",
on_disk_high: "rotate_logs_and_notify",
on_repeated_fail: "touch ~/JEQ_WORK/env/alert.flag"
}
}
},
stockfish: {
description: "CLI Stockfish for local chess analysis and as a deterministic compute test.",
install: {
source: "prebuilt-binary-or-apt",

termux_cmd: "pkg install -y stockfish",
usage_example: "stockfish bench 64"
},
role: "Deterministic engine for benchmarking, decision heuristics, and test workloads"
},
gemini_flash_cli: {
description: "CLI wrapper for Gemini Flash 2.5 style local inference (placeholder for compatible CLI).",
install_notes: "Provide binary named gemini-flash or wrapper that calls remote API when enabled.",
usage_example: "~/JEQ_WORK/bin/gemini-flash --model tiny --prompt '...' --max_tokens 128",
policy: "disabled by default; enable only with explicit API key in encrypted env"
},
copilot_cli: {
description: "CLI CoPilot GitHub-style helper for local orchestration and developer utilities.",
repo_example: "github:example/copilot-cli",
install_cmd: "git clone <repo> ~/JEQ_WORK/tools/copilot-cli && cd ~/JEQ_WORK/tools/copilot-cli && ./install.sh",
role: "Developer helper: scaffolding, logs, quick dispatch"
},
virtual_machine: {
description: "Lightweight VM/container option for isolated builds or running small cloud-bengkel services.",
options: ["UserLAnd", "proot-distro", "Termux QEMU (if supported)"],
recommended: {
proot_distro: "debian",
setup_cmds: [
pkg install -y proot-distro,
proot-distro install debian,
proot-distro login debian -- bash -lc 'apt update && apt install -y build-essential'

]
},
role: "Isolated build environment for llama.cpp builds or small service containers"
}
},
scripts: {
dispatch_sh: {
path: "~/JEQ_WORK/bin/dispatch.sh",
content: "#!/data/data/com.termux/files/usr/bin/bash\nMODE=\"\${1:-local}\"\n\nPROMPT=\"\${2:-Halo}\"\n\nif [\"\$MODE\" = \"local\"]; then\n ~/JEQ_WORK/bin/llama_main --model ~/storage/shared/AI_Models_User/phi-3-mini-4k-instruct-q4.gguf --threads 1 --n_ctx 512 --n_predict 64 --no-warmup --prompt \"\${PROMPT}\"\nelse\n echo \"Manual fallback requested. Check ~/JEQ_WORK/env/server.env for API_KEY and enable Agent_FALLBACK.\"\nfi"
},
watchdog_sh: {
path: "~/JEQ_WORK/bin/watchdog.sh",
content: "#!/data/data/com.termux/files/usr/bin/bash\n# Simple watchdog: check process and restart\nPROC_NAME=\"llama_main\"\nif ! pgrep -f \"\$PROC_NAME\" >/dev/null; then\n echo \"\$(date) - \$PROC_NAME not running, attempting restart\" >> ~/JEQ_WORK/logs/watchdog.log\n ~/JEQ_WORK/bin/llama_main --model ~/storage/shared/AI_Models_User/phi-3-mini-4k-instruct-q4.gguf --threads 1 --n_ctx 512 --n_predict 64 --no-warmup &\nfi"
}
},
security: {
secrets_policy: "Never store API keys in plaintext in Drive. Use local encrypted file with chmod 600.",
example_encrypt_cmds: [
pkg install -y openssl,
openssl enc -aes-256-cbc -salt -in server.env -out server.env.enc,
rm server.env && chmod 600 server.env.enc
],
restore_decrypt_cmd: "openssl enc -d -aes-256-cbc -in server.env.enc -out server.env"

},
backup_and_restore: {
rclone_example: {
install: "pkg install -y rclone",
config_note: "Create remote 'gdrive' with OAuth; restrict to Backup_Platform_AI folder",
backup_cmds: [
rclone copy ~/JEQ_WORK/logs gdrive:Backup_Platform_AI/logs --transfers 1,
rclone copy ~/JEQ_WORK/env gdrive:Backup_Platform_AI/env --transfers 1
],
restore_cmds: [
rclone copy gdrive:Backup_Platform_AI/env ~/JEQ_WORK/env --transfers 1,
chmod +x ~/JEQ_WORK/env/env_setup.sh && ~/JEQ_WORK/env/env_setup.sh
]
}
},
operational_spk: {
daily: [
Run watchdog check (cron or Termux:Tasker) every 5 minutes,
Sync logs to Drive manually or on Wi-Fi,
Rotate logs when >10MB
],
on_hard_reset: [
Login to Google Drive and rclone copy env and logs back,
Run ~/JEQ_WORK/env/env_setup.sh to restore environment,
Verify model presence in /sdcard/AI_Models_User
]
},

prompts: {
restore_prompt: "Restore environment and verify local model: list models, check llama_main process, run quick inference test.",
test_prompt_local: "Uji lokal: ringkas kalimat ini menjadi satu baris: Saya siap tunduk pada doktrin.",
fallback_prompt: "Fallback: buat ringkasan 200 kata dari teks panjang ini and return JSON with title and bullets."
},
deployment_checklist: [
1. termux-setup-storage,
2. mkdir -p ~/JEQ_WORK/{bin,logs,env},
3. copy llama-server to ~/JEQ_WORK/bin and chmod +x,
4. install rclone and configure gdrive remote,
5. place phi-3-mini-4k-instruct-q4.gguf in /sdcard/AI_Models_User,
6. install watchdog dependencies and place watchdog.sh in bin,
7. install stockfish via pkg and test bench,
8. optionally install proot-distro for VM builds,
9. configure Agent_FALLBACK only after encrypting API keys,
10. run dispatch.sh local test and archive logs to Drive
],
notes: {
stockfish_role: "Use for deterministic benchmarking and as a compute test harness.",
gemini_flash_note: "Gemini Flash CLI is a placeholder for a compatible local or remote CLI; enable only with explicit policy.",
copilot_cli_note: "CoPilot CLI is a developer helper; clone from GitHub and audit scripts before running."
}
}

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]